

SuryaWatch on AWS

Deployment guide for Agrim

Environmental Hacks 2026 · Team: Aryan Nirala, Agrim Chaturvedi, Prakhar Srivastava, Anshu Dhaka · Prepared 9 October 2026

In one minute

Aryan writes and tests the code and pushes it to GitHub. **Your job is to put it on AWS and keep it updated.** Everything is defined in one file (`template.yaml`), and one script (`scripts/deploy.sh`) deploys it. You run it in **AWS CloudShell**, the terminal built into the AWS website, so **nothing needs installing on your laptop.**

First deploy	About 45 minutes: 10 setup, 10 deploy, 25 checking.
Each update after that	About 5 minutes: two commands.
You need	The team AWS account login (region US East (N. Virginia) , us-east-1) and the GitHub repo link: https://github.com/aryanniralacsda25a07014-design/Suryawatch.git
You send Aryan	The web app link, the API link, and screenshots of each check in Part 4.

Two rules. Never paste AWS passwords or access keys into WhatsApp, chat, GitHub or any AI tool. You will never need access keys for this: CloudShell is already signed in as you. And **do not delete anything until the hackathon results are announced.**

1. What you are deploying

SuryaWatch is a web app for rooftop solar. **Plan** sizes a system and works out cost, subsidy and payback. **Watch** reads photos of the inverter display with AI and tells the owner whether low output is smog, dust or a fault, with email alerts every evening. The deploy script creates all of this:

AWS service	What it is used for
AWS Amplify Hosting	Hosts the web app (the public link).
Amazon API Gateway (HTTP API)	One web address the app sends every request to.
AWS Lambda (2 functions)	ApiFunction : all app logic. DailyCheckFunction : the 7:30 PM check.
Amazon DynamoDB (1 table)	Rooftops, readings, daily verdicts, cleanings, alert settings.
Amazon S3 (1 private bucket)	Inverter-display photos.
Amazon Bedrock (Amazon Nova 2 Lite)	Reads the display photos; answers questions in English and Hindi.
Amazon EventBridge Scheduler	Runs DailyCheckFunction every day at 7:30 PM IST.
Amazon SNS (1 topic)	Sends the alert emails.
AWS CloudFormation stack suryawatch	Holds all of the above, so it can be updated or removed together.

Cost: for a hackathon's traffic this fits inside the free tier and the account's new-user credits. The only pay-per-use part is Bedrock, at fractions of a rupee per photo. The zero-spend budget

in Part 2 emails you if anything starts costing money.

2. One-time setup (about 10 minutes)

1. Sign in to the AWS console. In the top-right corner set the region to **US East (N. Virginia) us-east-1**. Check this every time you open the console.
2. **MFA**: click the account name (top right) → Security credentials → Assign MFA device. Use an authenticator app on your phone.
3. **Budget alert**: search “Budgets” → Create budget → **Zero spend budget** → enter your email → Create.
4. **Check the AI model**: search “Bedrock” → Playgrounds → Chat / Text → Select model → **Amazon → Nova 2 Lite**. Attach any photo, ask “What do you see?”, and screenshot the answer for Aryan. Amazon Nova models are switched on by default, so there is no access form.
5. **Open CloudShell**: click the >_ icon in the top bar. A terminal opens at the bottom of the page. The first start takes about a minute.
6. In CloudShell, check the tools (all three are preinstalled):

Run in CloudShell

```
sam --version
git --version
python3 --version
```

Pasting into CloudShell: right-click → Paste, or Ctrl+Shift+V. If a box warns about pasting several lines, press *Paste*. CloudShell closes after about 20–30 minutes of no use. Just reopen it: your files in the home folder are kept.

3. First deploy (about 10 minutes)

Copy these into CloudShell one block at a time.

Step 1 · Download the code from GitHub

```
cd ~
git clone https://github.com/aryanniralacsda25a07014-design/Suryawatch.git suryawatch
cd suryawatch
ls
```

You should see backend frontend scripts template.yaml tests If git asks for a username or password, the repo is private: ask Aryan to make it public (GitHub → Settings → General → Danger zone → Change visibility).

Step 2 · Deploy everything

```
bash scripts/deploy.sh
```

The script does four things and prints each step:

Step it prints	What happens	Time
1/4 Running tests	Runs the automated tests. If they fail, it stops and deploys nothing: send the output to Aryan.	10 s
2/4 Deploying backend	SAM packages the Python code and CloudFormation creates or updates every service in Part 1. You will see a table of resources being created.	3-6 min first time
3/4 Packaging web app	Points the web app at the new API address.	5 s
4/4 Publishing web app	Uploads the web app to Amplify Hosting.	1 min

At the end it prints two links. **Copy both and send them to Aryan.**

What success looks like (your letters will differ)

```
Done.
API:      https://abc123xyz.execute-api.us-east-1.amazonaws.com/health
Web app:  https://main.d1abc2defgh.amplifyapp.com   (live in about a minute)
```

Lost the links? This prints them again

```
aws cloudformation describe-stacks --stack-name suryawatch --region us-east-1 \
--query "Stacks[0].Outputs" --output table
```

If it fails. If step 2/4 ends in an error such as CREATE_FAILED or ROLLBACK_COMPLETE, scroll up to the first red “CREATE_FAILED” line: the reason is printed next to it. Send a screenshot to Aryan. The fix for a stack stuck in ROLLBACK_COMPLETE is in Part 7.

4. Check that everything works (about 25 minutes)

Do these in order and screenshot each result for Aryan. Replace API with the API link (without /health).

#	Check	How	Expected result
1	API is up	In CloudShell: <code>curl API/health</code>	<code>"ok": true, "stage": 3, "storage": "aws", "alerts": "sns"</code>
2	Web app opens	Open the web app link on laptop and phone.	The SuryaWatch page, with no yellow “not connected” banner at the top.
3	Plan works	Plan tab: click a house, enter roof area 800 sq ft and 400 units, press Plan my solar.	System size, cost, payback and a monthly chart.
4	Demo data	In CloudShell run the sample-data command below.	Prints 10 days and a link. Open it: verdicts, charts, green calendar, cleaning effect.
5	AI photo reading	Watch tab → Upload photos → a real inverter-display photo.	Numbers filled in, with a note “Read by AI (... confidence)”. It must not say TEST MODE.
6	AI helper	Under a verdict, tap a question. Then switch the language to Hindi (the option next to English) and tap one.	A short answer marked “Amazon Bedrock”, in English and in Hindi.
7	Email alerts	Watch tab → Email alerts → your email → Get alerts. Open the email from “AWS Notifications” and click <i>Confirm subscription</i> . Then press <i>Send today's check now</i> .	An email with subject “SuryaWatch: ...” within a minute (check spam).
8	Evening schedule	Console → search “EventBridge” → Scheduler → Schedules.	One schedule for DailyCheckFunction, <code>cron(30 19 * * ? *)</code> , Asia/Kolkata.

Check 4 · Load the labelled demo rooftop into the cloud (about 1 minute)

```
python3 scripts/sample_data.py --api API
```

5. Updating after Aryan pushes changes (about 5 minutes)

Whenever Aryan says “new version pushed”, run this in CloudShell. Existing data (rooftops, readings, alert emails) is kept.

Every update

```
cd ~/suryawatch
git pull
bash scripts/deploy.sh
```

Then hard-refresh the web app (Ctrl+Shift+R) and repeat any check Aryan asks for. If `git pull` complains about local changes, run `git checkout -- .` and then `git pull` again. You should never edit files in CloudShell.

6. Show AWS in the demo video

The rules say the 3-minute video must **show** AWS being used; naming it is not enough. Screen-record about 5–10 seconds of each of these pages after the checks in Part 4 have created real data. Send the clips to whoever edits the video.

Show this	Where in the console
The whole stack	CloudFormation → Stacks → suryawatch → Resources tab (lists every service).
The two Lambda functions	Lambda → Functions → the suryawatch ApiFunction and DailyCheckFunction.
Readings stored	DynamoDB → Tables → the suryawatch table → Explore table items.
Photos stored	S3 → Buckets → the suryawatch photo bucket → photos/ folder.
AI model	Bedrock → Model catalog → Amazon Nova 2 Lite (or the Playground screenshot).
Evening schedule	EventBridge → Scheduler → Schedules → the 7:30 PM IST schedule.
Alert emails	SNS → Topics → SuryaWatch → Subscriptions (status Confirmed), plus the email in your inbox.
Hosting	Amplify → All apps → suryawatch → the main branch with its live link.
Live logs (optional)	CloudWatch → Log groups → the ApiFunction log group, while someone uploads a photo.

7. If something goes wrong

Problem	Fix
sam: command not found	You are not in CloudShell. Open it from the >_ icon in the AWS console.
git asks for a password	The repo is private. Ask Aryan to make it public, then clone again.
“TESTS FAILED - fix before deploying”	Do not deploy. Send the printed output to Aryan.
Stack shows ROLLBACK_COMPLETE	A first deploy failed and AWS undid it. Run <code>sam delete --stack-name suryawatch --region us-east-1</code> , answer <code>y</code> , then run <code>bash scripts/deploy.sh</code> again. Send Aryan the first <code>CREATE_FAILED</code> reason.
“Upload failed (HTTP ...)” in step 4/4	Run <code>bash scripts/deploy.sh</code> again (safe to repeat).
Yellow “not connected” banner on the web app	Step 3/4 did not run. Run the deploy script again, then hard-refresh (Ctrl+Shift+R).
Photo note says “No Bedrock model could read the photo”	The app already tries three Amazon Nova models. Check the region is <code>us-east-1</code> , and that Nova 2 Lite works in the Bedrock Playground (Part 2). Send Aryan the logs (command below).
“ThrottlingException” or “Too many requests” in the logs	New AWS accounts start with low AI limits. Wait a minute and retry. If it persists: Service Quotas → Amazon Bedrock → search “Nova” → request an increase.
No alert email	Confirm the subscription from the AWS email first (check spam). Then check SNS → Subscriptions shows “Confirmed”, and press <i>Send today's check now</i> again.
Anything else	Run the log command below, reproduce the problem in the app, and send Aryan the red lines.

Live logs

```
sam logs --stack-name suryawatch --name ApiFunction --region us-east-1 --tail
# press Ctrl+C to stop. For the evening check use --name DailyCheckFunction
```

8. After the results are announced (not before)

To remove everything and stop all costs: empty the photo bucket, then delete the stack.

Clean-up, only after results

```
cd ~/suryawatch
B=$(aws cloudformation describe-stacks --stack-name suryawatch --region us-east-1 \
  --query "Stacks[0].Outputs[?OutputKey=='PhotoBucketName'].OutputValue" --output text)
aws s3 rm s3://$B --recursive
sam delete --stack-name suryawatch --region us-east-1
```

Quick reference

When	Command (in CloudShell)
First time	<code>cd ~ && git clone https://github.com/aryanniralacsd25a07014-design/Suryawatch.git suryawatch && cd suryawatch && bash scripts/deploy.sh</code>
Every update	<code>cd ~/suryawatch && git pull && bash scripts/deploy.sh</code>
Show the links	<code>aws cloudformation describe-stacks --stack-name suryawatch --region us-east-1 --query "Stacks[0].Outputs" --output table</code>
Demo data	<code>python3 scripts/sample_data.py --api API</code>
Live logs	<code>sam logs --stack-name suryawatch --name ApiFunction --region us-east-1 --tail</code>